

Build Multi-Tenant SaaS Platform with Project & Task Management

Mandatory Task

Back

Domain

- Backend Development
- Frontend Development

Skills

- API Development
- API Documentation
- Artificial Intelligence
- Database Management
- Email Security
- Full-Stack Development
- Management
- Scalability
- Solution Architecture
- System Design
- Authentication & Authorization

Difficulty

Hard

Tools

- Docker
- Express.Js
- Git
- JWT
- Node.Js
- PostgreSQL
- Postman
- React
- Swagger
- bcrypt

🎯 Pending Submission

TIME REMAINING

Deadline: 27 Dec 2025, 04:59 pm. Please submit your work before the deadline.

1d

- Overview
- Instructions
- Resources
- Submit

Submission Instructions

Submit a complete, dockerized multi-tenant SaaS application with full documentation.

Required Submissions:

1. GitHub Repository (Public)

🛠 Report Issue

- All source code for backend API and frontend application

partnr

- Minimum 30 commits with meaningful commit messages
- Repository must be public and accessible

2. Dockerized Application (MANDATORY)

- **Docker Compose Configuration:** A complete `docker-compose.yml` file in the project root that defines ALL THREE services (database, backend API, and frontend)
- **Frontend Containerization (MANDATORY):** Frontend MUST be containerized. All three services (database, backend, frontend) MUST start with a single command:
`docker-compose up -d`
- **Fixed Port Mappings (MANDATORY):**
 - Database: Port `5432` (external) → `5432` (internal)
 - Backend: Port `5000` (external) → `5000` (internal)
 - Frontend: Port `3000` (external) → `3000` (internal)
- **Service Names (MANDATORY):**
 - Database service MUST be named: `database`
 - Backend service MUST be named: `backend`
 - Frontend service MUST be named: `frontend`
- **Backend Dockerfile:** A properly configured `Dockerfile` for the backend application with optimized multi-stage builds
- **Frontend Dockerfile:** A properly configured `Dockerfile` for the frontend application
- **Service Configuration:** All services must be properly configured with environment variables, dependencies, and networking
- **Environment Variables:** All environment variables MUST be present in the repository (either in `.env` file committed to repo, or directly in `docker-compose.yml`). Evaluation script needs access to all environment variables.
- **Volume Management:** Database data persistence using Docker volumes
- **Database Initialization (MANDATORY - Automatic Only):** Database migrations MUST run automatically when backend service starts
- **Seed Data Loading (MANDATORY - Automatic Only):** Seed data MUST load automatically after migrations complete
- **No manual commands allowed** - everything must be automatic via Dockerfile entrypoint or startup script
- **Seed Data Requirements:** You can choose your own seed data, but MUST include minimum:
 - At least one `super_admin` user
 - At least one tenant with `tenant_admin`
 - At least one regular user per tenant

- At least one project per tenant

partnr

Dashboard Skill Graph Profile GPP

- **Documentation:** Clear instructions in README.md on how to run the application using `docker-compose up -d`
- **Testing:** The application must be fully functional when started with Docker Compose (all services start successfully, database migrations run, seed data loads, health check endpoint responds, frontend is accessible)
- **Why Docker is Required:** Dockerization ensures consistent environments, easy evaluation, dependency management, and reproducibility. Without Docker, the application cannot be evaluated.

3. Documentation Artifacts

- README.md : Complete project documentation with setup instructions (including Docker setup), architecture overview, and API documentation links
- docs/research.md : Multi-tenancy analysis, technology stack justification, security considerations (minimum 1700 words total)
- docs/PRD.md : Product Requirements Document with user personas, 15+ functional requirements, 5+ non-functional requirements
- docs/architecture.md : System architecture diagram, database ERD, complete API endpoint list
- docs/technical-spec.md : Project structure and development setup guide (must include Docker setup instructions)
- docs/API.md or Swagger/Postman: Complete API documentation for all 19 endpoints
- docs/images/system-architecture.png : High-level system architecture diagram
- docs/images/database-erd.png : Entity Relationship Diagram showing all tables and relationships

4. Submission JSON File (MANDATORY)

- File: submission.json in repository root
- **Purpose:** Contains test credentials for automated evaluation
- **MUST include:** ALL seed data credentials (super admin, tenant admins, users, etc.)
- **Format:** See detailed format specification in task documentation
- **Note:** Documentation, demo video, and tech stack are collected separately in the submission form

5. Demo Video (YouTube)

- Upload to YouTube as Unlisted or Public
- Duration: 5-12 minutes

- Must include: Introduction, architecture walkthrough, running application demo (tenant registration, user management, project & task management, multi-

partnr

• Submit YouTube link separately in submission form (not in submission.json)
- DashboardSkill GraphProfileGPP

Evaluation Focus:

- Complete data isolation between tenants
- Proper authentication and authorization implementation
- All 19 API endpoints functional and properly secured
- Frontend pages with role-based access control
- **Docker configuration and containerization (MANDATORY)**
- Database initialization and seed data loading
- Health check endpoint functionality
- Comprehensive documentation quality

Evaluation Overview

Your submission will be evaluated through multiple dimensions to assess both functional completeness and code quality.

Functional Evaluation: We will test all 19 API endpoints to verify they work correctly, return proper status codes, and enforce authentication and authorization. We will verify data isolation by attempting cross-tenant access scenarios. The frontend will be tested for all required pages, role-based UI visibility, and responsive design. We will verify Docker setup by running `docker-compose up -d` and testing the health check endpoint.

Docker Configuration Evaluation: We will verify that your application is properly dockerized with a working `docker-compose.yml` file and `Dockerfile(s)` for all three services (database, backend, frontend). We will test that all services start successfully using `docker-compose up -d` with fixed port mappings (database: 5432, backend: 5000, frontend: 3000), that database migrations run correctly (automatically or via documented command), that seed data loads properly (automatically or via documented command), and that the application is fully functional in the containerized environment. Docker configuration is mandatory for evaluation.

Database Initialization Evaluation: We will verify that the database is properly initialized with migrations and seed data. We will read test credentials from your `submission.json` file and check that the documented seed data exists in the database after initialization. We will test login with credentials from your `submission.json` to verify seed data is correctly loaded.

Code Quality Review: Your codebase will be analyzed for organization, security best practices, error handling, and architectural patterns. We will review database

partnr

Dashboard Skill Graph Profile GPP

Documentation Assessment: All documentation artifacts will be reviewed for completeness, clarity, and technical depth. Research documents will be evaluated for analysis quality and justification of design decisions. Architecture diagrams will be assessed for clarity and completeness. API documentation will be checked for accuracy and comprehensiveness. Docker setup instructions in README.md will be verified for clarity and completeness.

Human Review: Visual aspects including UI/UX quality, responsive design, and user experience will be evaluated. Your demo video will be reviewed for presentation quality and feature demonstration. Architecture diagrams and documentation will be assessed for clarity and professional presentation. Docker configuration and containerization approach will be evaluated for quality and best practices.

Security & Best Practices: We will verify proper password hashing, JWT security, input validation, SQL injection prevention, and proper error handling. Audit logging implementation and data isolation mechanisms will be thoroughly examined. Docker security best practices (non-root users, minimal base images, proper secret management) will also be assessed.

[About Us](#) [Contact Us](#) [Privacy Policy](#) [Terms and Conditions](#)

All rights reserved. Copyright, Partnr 2025–26

